

Python Interpreter Hot-Path Policy

Version: mdstats 0.19.72a0

Status: implemented for registered numerical kernels; package-wide audit active

1. Purpose

mdstats may orchestrate scientific calculations in Python, but it must not execute dense numerical work one physical element at a time through the Python interpreter. Large loops over trajectory frames, atoms, grid nodes, Gaussian contributions, mesh vertices, mesh faces, or candidate pairs are prohibited in registered numerical hot paths unless the operation is intrinsically irregular and no compiled implementation is available.

The default implementation hierarchy is:

1. NumPy broadcasting, indexing, reductions, sorting, and batched matrix multiplication;
2. SciPy compiled kernels, sparse matrices, FFTs, and spatial algorithms;
3. established compiled libraries such as scikit-image marching cubes and fast-simplification;
4. a bounded coarse Python loop over fields, components, tiles, or chunks;
5. a compiled mdstats extension for irregular algorithms that remain dominant.

`numpy.vectorize` and `numpy.frompyfunc` are not accepted as numerical acceleration because they retain Python-call dispatch.

2. Loop classification

2.1 Prohibited dense loops

A Python `for` or `while` loop is prohibited when its iteration count scales directly with any of:

- trajectory frames multiplied by atoms;
- grid nodes or voxels;
- Gaussian source-stencil contributions;
- mesh vertices, faces, or edge occurrences;
- neighbor candidate pairs;
- rows of fixed-width packed numerical records.

These operations must be delegated to a compiled array or numerical kernel.

2.2 Permitted orchestration loops

Python loops are permitted when they operate over a small or deliberately bounded outer structure, for example:

- density fields or chemical species;
- three Cartesian axes;
- render tiles or compute tiles whose inner arrays are processed by compiled kernels;
- memory-bounded chunks;
- connected components or topology states;
- user-facing trace construction.

The loop body must not contain another large elementwise Python loop.

2.3 Irregular graph algorithms

Primitive-ring enumeration, topology projection, symmetry discovery, natural-tiling search, and exact periodic graph certification are irregular combinatorial algorithms. NumPy broadcasting is not automatically appropriate. Their policy is:

1. retain clear Python orchestration while problem sizes are small;
2. profile representative large cases;
3. move dominant queue, adjacency, path, or exact-predicate kernels to a compiled extension when runtime becomes material;
4. preserve deterministic ordering and exact integer-periodic semantics across implementations.

3. Registered interpreter-free kernels

The following functions are guarded against reintroduction of Python `for` or `while` loops:

- `pack_local_indices()`;
- `unpack_local_bitset()`;
- `bitset_popcounts()`;
- `_reverse_source_target_csr()`;
- `_precise_vertices_and_keys()`.

A focused AST regression test enforces this boundary.

4. Implemented changes

4.1 Packed bitsets

The former implementation iterated over Python integers to set, decode, and count bits. The new implementation uses:

- `numpy.bitwise_or.at` for packed-bit construction;
- `numpy.unpackbits(..., bitorder="little")` for decoding;
- a compiled uint8 lookup-table reduction for row popcounts;
- `int.from_bytes()` and `int.to_bytes()` for packed integer conversion.

The normative little-endian uint64 bit layout is unchanged.

4.2 Sparse CSR transpose

The reverse target-to-source map formerly inserted every edge through nested Python loops. It now constructs source IDs with `numpy.repeat`, sorts `(target, source)` records with `numpy.lexsort`, and builds CSR ranges with `numpy.bincount` and `numpy.cumsum`.

4.3 Tiled contour vertex recovery

Exact logical-edge reconstruction for marching-cubes vertices formerly performed scalar Python work per vertex. Endpoint construction, volume gathers, interpolation, clipping, and coordinate recovery are now vectorized. Python objects are created only for immutable cross-tile hash keys.

Final key ordering, cyclic face canonicalization, and face sorting use NumPy sorting and indexed gathers.

4.4 Density planning

Per-source support upper bounds and terminal block extents are evaluated as arrays. The planner no longer loops through every active source block merely to multiply independent axis counts or determine maximum brick dimensions.

4.5 Frame-dependent cell transforms

Frame-by-frame $N \times 3$ matrix multiplication has been replaced with NumPy broadcasted `matmul`:

```
cartesian = fractional @ frame_cells
```

This is preferred over `einsum` for this particular batched `(..., 3) @ (..., 3, 3)` operation because measured `einsum` performance was worse.

5. Stage-2 status and remaining boundary

The former P0/P1 items for cell-list candidate expansion, tiled-mesh reconciliation, bond-angle accumulation, support-atlas merging, and fragmented direct realization are implemented in 0.19.72a0. The implemented kernels are now part of this policy rather than a separate stage specification. They include:

- batched cell-list candidate-bin expansion and exact metric-box pruning;
- ragged bond-angle pair generation and compiled histogram reduction;
- fixed-width support-atlas merging and SciPy sparse connected components;
- global sort/unique mesh reconciliation with interpreted clipping restricted to boundary-crossing triangles;
- bounded ragged scheduling for fragmented direct sparse realization.

Focused benchmark evidence remains in `audits/release/interpreter_hotpath_stage2_benchmarks.json`.

The remaining high-priority work is intentionally restricted to irregular graph and topology algorithms: primitive-ring enumeration, natural tiling, periodic symmetry discovery, and exact periodic-cell certification. These algorithms must be profiled on representative large systems before a compiled extension is introduced. They must not be converted into large dense tensors merely to remove Python syntax.

A future compiled graph kernel must preserve exact integer image shifts, deterministic canonical ordering, bounded search limits, and the existing immutable CSR/array public contracts.

6. Verification requirements

Every hot-path replacement must provide:

- exact or tolerance-qualified numerical equivalence;
- deterministic ordering equivalence;
- a focused correctness test;
- a representative microbenchmark against the replaced implementation;
- an AST regression test when the intended kernel must remain interpreter-free;
- memory-use accounting for any new vectorized temporary arrays.

A speedup is not accepted if it changes scientific resolution, estimator semantics, periodic image conventions, or resource-budget enforcement.

Implemented dense-kernel registry

The following paths are protected against reintroduction of elementwise Python loops:

Module responsibility	Required implementation form
packed-bitset construction and decoding	NumPy bitwise operations and byte lookup tables
reverse sparse CSR construction	array sorting, counting, and prefix sums
exact contour-edge recovery	vectorized array arithmetic
frame-dependent 3×3 transforms	broadcasted <code>numpy.matmul</code>
cell-list candidate expansion	bounded array batches
metric-stencil active-set evaluation	fixed 27-pattern loop over vector batches
bond-angle accumulation	ragged templates and compiled reductions
support-atlas union	fixed-width records and sparse graph routines
tilled-mesh welding	global sort/unique assembly

Module responsibility	Required implementation form
fragmented direct realization	bounded vector chunks

A static scan is a discovery tool, not proof of a defect. A rewrite is accepted only when correctness tests pass and representative benchmarks show a material gain.